Call Log Merger: Algorithms

The program stores each call as a node in a singly linked list. It reads two call logs, appends each call to its log, joins the two lists, and prints the merged result. Each function is shown as language-neutral pseudocode.

1. Data Structure

Each call is a Node with **name**, **duration**, and **next**. next points to the following call, and the last node has next = NULL. A list is identified by its head pointer, which is NULL when the list is empty.

2. Algorithms

createNode(name, duration)

```
newNode <- allocate new Node
newNode.name <- name
newNode.duration <- duration
newNode.next <- NULL
return newNode
```

insertEnd(head by reference, name, duration)

```
newNode <- createNode(name, duration)
if head == NULL:
    head <- newNode
    return
temp <- head
while temp.next != NULL:
    temp <- temp.next
temp.next <- newNode
```

mergeLists(head1, head2)

```
if head1 == NULL:
    return head2
temp <- head1
while temp.next != NULL:        # find tail of list 1
    temp <- temp.next
temp.next <- head2
return head1
```

display(head)

```
while head != NULL:
    print head.name, head.duration, "min"
    head <- head.next
```

```
main()

list1 <- NULL; list2 <- NULL
read n
repeat n times:
    read name, duration
    insertEnd(list1, name, duration)
read m
repeat m times:
    read name, duration
    insertEnd(list2, name, duration)
list1 <- mergeLists(list1, list2)
print "Merged Call Log:"
display(list1)
return 0
```

3. Complexity Summary

Function	Time	Notes
createNode	O(1)	Constant allocation and assignments
insertEnd	O(n)	Walks to the tail of the list
mergeLists	O(n)	Walks list 1 to its tail, then links list 2
display	O(n)	Visits every node once
Whole program	O(n + m)	Space O(n + m) for the nodes